

Unit 06: Unformatted and Formatted I/O

CONTENTS

Objectives

Introduction

6.1 Input/Output Function

6.2 Formatted Input/Formatted Output

6.3 printf()

6.4 Escape Sequences

6.5 scanf()

6.6 Sign (*)

6.7 Reading and Writing a Character

6.8 gets() and puts() Functions

Summary

Keywords

Self Assessment

Answers for Self Assessment

Review Questions

Further Readings

Objectives

After studying this unit, you will be able to:

- Describe formatted input/output
- Explain the input/output function
- Know how to read and write a character

Introduction

If a user is unable to communicate with a computer programme, it is unlikely that the programme would be useful. The software must read input values from the user console and generate the expected useful output to the user in most programming assignments.

C is unique in that, unlike other high-level languages, it does not have any built-in I/O statements in its syntax. We'll look at how C handles different I/O operations in this unit. We'll also hear about the standard library functions in C, as well as the header files that go with them.. This unit is not intended to be a complete treatment of these topics, but it provides enough information so that you can start writing real programs.

6.1 Input/Output Function

Data is transferred to peripheral devices such as a computer, keyboard, printer, or secondary storage through I/O operations. Since C has no built-in support for receiving data from input devices (such as a keyboard) or sending data to output devices (such as a monitor), all I/O operations must depend on library functions like printf (). A library is a set of predefined functions

contained in one or more files. The creators of the C compiler created a library named "C standard library" that contains many standard I/O functions.

Depending on the compiler and necessary functions, various methods may be used to access these library functions from the standard library. Some compilers scan libraries for named functions automatically. Most of the time, the programmer must specifically state the library file name during the linking process, resulting in the executable programme only containing the necessary functions.

These functions are stored in a special file with the extension.h (such as `stdio.h`), which is referred to as a header file. The `#include` compiler directive can be used to insert a header file into a C programme file, as shown below.

```
#include<stdio.h>
```

The `#include` directive tells the compiler to read the `stdio.h` (standard input output header file) file and substitute this line with the contents of that file. Similarly, in order to use other predefined functions, their respective header files must be included in a programme such that the function declaration can be accessed by the programme.

C has a rich set of standard I/O library functions. However, these I/O functions are not the part of C's formal definition. C's standard library function for I/O can be broadly divided into the following categories:

1. Port I/O functions
2. Disk I/O function
3. Console I/O function

The Port I/O function handles various I/O operators on various ports, such as a mouse port or a printer port. The scope of this text does not include a thorough examination of the port I/O functions. The disc I/O functions are used to manipulate files on secondary storage devices such as a floppy disc or a hard disc. Since files are stored in secondary storage, such as discs, disc I/O functions are nothing more than file handling functions.

The term "console" refers to the regular input and output devices in its broadest sense. These basic input or output devices, which are also known as the keyboard and display by default, are dealt with by console I/O functions. These functions take keyboard input and display the results on the computer.

C takes all input and output as stream of characters. A stream is nothing,

but a series of bytes. C language treats all streams equally i.e., whether a program gets input from the keyboard, a disk file or a modem, it consider it as only a stream of characters.

Different streams are used to represent different kinds of data flow. In C, there are three streams associated with console I/O operations.

1. ***stdin***: A stream that supplies data to the program i.e., standard input, usually from the keyboard.
2. ***stdout***: A stream that receives data from the program i.e., standard output; usually to the monitor.
3. ***stderr***: A stream used to keep error messages separate from program's output i.e., standard error; usually points to your terminal screen.

For conducting console I/O operations, C offers a number of functions. These functions allow information to be transferred between the computer's standard input and output devices (i.e., keyboard and monitor). Only a few of them allow you to format input and output operations. Some of them, on the other hand, do not allow you to monitor the format of I/O operations.

From this aspect, console I/O operations can be further categories as:

1. Unformatted console I/O functions
2. Formatted console I/O functions

It is important to include the standard I/O library header file in order to use these functions. The declarations for these functions can be found in the `stdio.h` header file. As a result, before using these console I/O features, always include the header file `stdio.h` in your C programme.

6.2 Formatted Input/Formatted Output

In this category, we have functions that allow input and output operations to be performed in a fixed format. Formatting of I/O operators deals with some of the following issues like:

1. How much field width is required to display the various values on the monitor?
2. How many decimal places are required to display the fractional part of a real number?
3. Should data values be left aligned or right aligned, and how much?
4. How much space between two data values is to be given?
5. How various type of data i.e. integer, character, and string can be used together I/O operators, etc.

Printf() and scanf() are the two most commonly used functions for formatted I/O. (). The printf() function is used to view formatted data items on a standard output device, such as a monitor, while the scanf() function is used to read formatted data input from a standard input device, such as a keyboard. However, due to their greater complexity, both functions are slower than the previous classes of functions. These functions are specified in the stdio.h header file and return EOF if an error or the end of the file occurs. Let's take a closer look at each of these functions one by one.

6.3 printf()

The printf() is one of the most important and useful functions to display data on monitor. We have seen the use of printf() for printing messages in the various example given previously in this book. For example, the statement

```
printf(" this section will discuss printf() in detail");
```

will simply print this message on the monitor. Besides these text messages, a program frequently requires numeric values and the value of other variables to be displayed on the screen.



Example: In order to print the sum of two numbers say a and b, in a new line along with some identifying text, the printf() will take the following form:

```
printf("\n the sum of %d and %d is %d.", a, b, c);
```

If the value of a and b is 5 and 6 respectively, then the output would be as follows:

The sum of 5 and 6 is 11.

Undoubtedly, a little more complicated than printing a simple message. Before getting the detail of the various sections of this printf(), let's discuss the general format of a printf(), shown below:

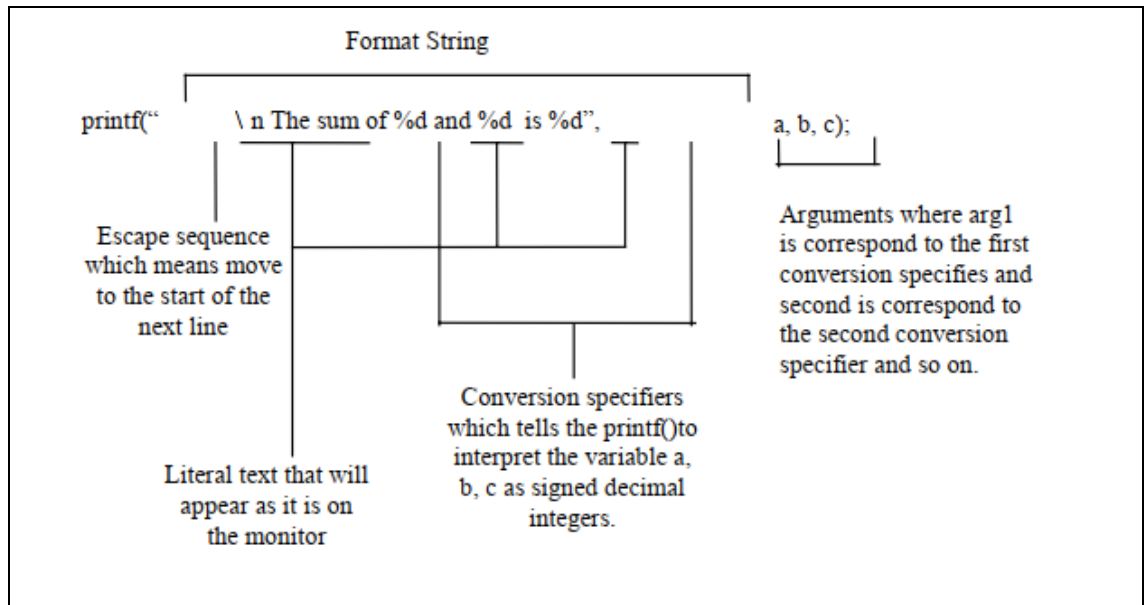
```
printf("Format string", arg1, arg2,..., argn);
```

Where format string refers to a string enclosed in double quotes that contains formatting information and arg1, arg2, ..., argn are arguments (may be constants, variable, or other complex expressions) whose values are formatted and printed according to the specification of the format string.

The format string in a printf() contains the format specifier that defines how the output is formatted. Following are the three possible components of a format string:

1. Literal text that is simply printed as entered in the format string.
2. An escape sequence that begins with a \ (backslash) sign, provides special formatting control.
3. A conversion specifier that begins with a % sign and followed by a single character, that tells printf() how to interpret the arguments being used. To understand the various sections

of previously used printf() statement,



The output of given statement if the value of a and b is 5 and 6 respectively, would be as:

The sum of 5 and 6 is 11

Let's see how this output is evaluated:

The printf() function reads the format string from left to right and sends the characters that follow to the standard output unit. It takes action as soon as it encounters the (backslash) (that is, the signal of the existence of an escape sequence). When it comes across the percent (conversion specifier) symbol, it grabs the corresponding argument and prints its value in the format defined. When the closing pair of double quotes is encountered, the process comes to an end.

In our case, the first character after the opening pair of double quote is \ , followed by a character n, so the effect of \n will take place i.e., output of the coming characters will start from the starting of the next line. Output up to this stage appears as:

The sum of

Then comes the character % followed by character d (that is, the indication to treat the corresponding variable as assigned decimal integer), so it picks up the variable a and will print

its value on the screen. At this stage the output will be looking like as:

The sum of 5

In the same manner, this process will go on until there comes an end point of format string. The final output would be appears as:

The sum of 5 and 6 is 11

The following program will help you to understand the concept more closely as it uses the printf() statement to print the result of the calculations.

Write a program to print the sum of two numbers.

Program:

```
#include<stdio.h>
```

```
void main()
```

```
{
```

```
int a, b, c;
```

```

a = 5;
b = 6;
c = a + b;
printf("\n sum = %d", C); /* will print the desired */
/* result on the monitor */
}

```

Output:

Sum = 11

Format Specifiers

%d -- displays a decimal (base 10) integer

%i -- used with other specifiers to indicate a "long"

%e -- displays a floating point value in exponential notation

%f -- displays a floating point value

%c -- displays a single character

%s -- displays a string of characters



Example: Formatted Output with printf ()

Int a;

Float b;

```
scanf("%d %f", &a, &b);
```

```
printf("the value of a= %d and b= %f \n", a, b);
```

6.4 Escape Sequences

Escape sequences, as previously mentioned, are used to control the position of output by shifting the monitor's cursor. Any character preceded by a backslash is meant to be treated as an escape sequence. The backslash informs the compiler that this is a special character constant of different significance than printf ().

Character Constant	Meaning
"X"	The character X
"\X"	The character that follows this is in hexadecimal
"n"	The character n
"\n"	New line

Program shows the usage by some of the frequently used escape sequences

```
#include<stdio.h>
```

```
void main()
```

```
{
```

```
printf("\n 1..\t2..\t3...\n");
```

```
printf("The question is, \" said Humpty Dumpty,\"which is to be masterthat\'s all.\");
```

```
}
```

Output:

1.. 2.. 3....

"The questions is, "said Humpty Dumpty," which is to be master-that's all".

6.5 Scanf()

As previously stated, in order to write an interactive application, certain statements in the programme must be able to collect data from the user. We provided a few functions in this sense, such as `getchar()`, `gets()`, and `getch()`, among others. A programmer, on the other hand, needs more flexibility in terms of:

1. Read the data from keyboard according to a specified format
2. Instruct the compiler to receive the particular type of value from the keyboard. For instance, integer value or floating point value.

Instruct the compiler to read the specified number of digits of a given number. Reading mixed data types from the keyboard using single function.

But the use of above mentioned functions is restrict with the character values only. There is a need of more flexible and general function that could address the problems mentioned above.

`scanf()`, the complement of the `printf()`, can actually be used to read the different type of data from the keyboard in a specified format. Due to what, it is referred to as formatted input functions.

Like `printf()`, `scanf()` also uses a format string to describe the format of the input, but with some little variations as given below:

1. It doesn't allow escape sequences in the format string.
2. It requires a special operator `&` called as "address of" to be prefix with the variable identifiers.

So, a `scanf()` takes the following from:

```
scanf(" format string", arg1, arg2,.....argn);`
```

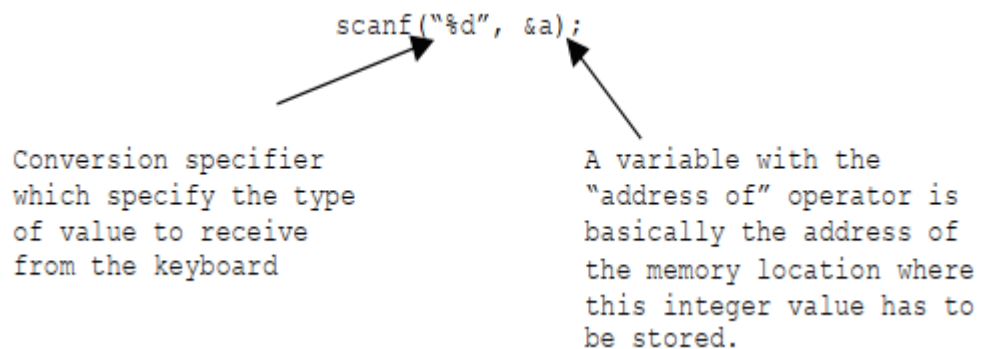
Where format string contains the formatting information by using which the data is to be entered and `arg1`, `arg2`,..., `argn` are the arguments (normally variables preceded by an ampersand `&`) specify the address of location where the data is stored. Both the section i.e., format string and arguments (within itself also) must be separated by commas.

The format string in a `scanf()` describe the format of the input and it may contain:

1. Conversion specifiers as in the `printf()` functions.
2. White space characters i.e., tabs, blanks, and newlines.
3. Other characters than white spaces, that are matching characters and asteric

6.6 Sign (*)

To have a better understanding of the concept, consider the following statement:



In layman's terms, the preceding statement is an instruction to the compiler to obtain an integer value from the keyboard and store it in a variable called a. Where an is an integer variable that was previously declared. Following this argument, the value stored in the variable can be used anywhere for any reason. Let's take a closer look at the different components of this sentence.

Conversion Specifier in scanf()

As mentioned earlier, a conversion specifier instructs the scanf() to convert the input stream of binary data coming from the keyboard into the data type specified by the conversion character.

For instance, integer in case of %d as it utilizes the same conversion specifiers as the printf() except one i.e., [...]. More than one conversion specifier can be used in a single scanf() to read more than one value.

In such a case, corresponding variables each preceded by & must include in the same statement.



Example: The statement.

```
scanf(" %d %d", &a, &b);
```

Will read two integer values from the keyboard, first value will be assigned to a and second to b.

White Spaces in scanf()

When several variables are entered in a single scanf(), the white space character may be used to distinguish them (i.e., blank space, tabs or new line character). White spaces in the format specifier are not taken into account. As a result, they will be read in the input data but ignored. Example: Consider the following statement once again:

Single space



```
scanf(" %d %d", &a, &b);
```

During runtime, in response to the above statement, you could enter

10 20



single space

As white space can be a tab, so you can also enter as

10 20

tab

A white space can be in the form of newline character, so you could also enter

10 ↵

20

As white spaces are required in input stream, they just can be used to identify the end of each input value.

For real time experience, consider the following program which demonstrate the usage of scanf() to read integer values from the keyboard. This program will accept two numbers from the user and will print their sum on the monitor

**Lab Exercise:** Program: input function

```
#include<stdio.h>

int main()
{
    int var1;
    printf("Enter value of var1");
    scanf("%d",&var1);
    printf("Value of Var1 is = %d",var1);
    return 0;
}
```

**Lab Exercise:** Program: To find radius of circle

```
#include<stdio.h>

void main()
{
    float r, a, c;
    const float pi = 3.14;
    printf("\n Enter the radius of a circle:");
    scanf(" %f", &r);
    a = pi * r * r; /* as r power 2 = r * r */
    c = 2 * pi * r;
    printf("\n Area = %f", a);
    Printf("\n circumference = %f", c);
}
```

**Lab Exercise:** Program : lower Case to upper case

```
#include<stdio.h>

void main()
{
    char ch;
    printf("\n Enter any character in lower case:");
    scanf("%c", &ch);
    printf(" \n The typed character in upper case is %c", ch);
}
```

Output:

Enter any character in lower case: h

The typed character in upper case is: H



Task: If execution is not provided with the proper input, result may be unexpected

6.7 Reading and Writing a Character

Unformatted console I/O functions don't allow input and output to be formatted as per the user requirements. In this category, we have:

1. Character I/O functions
2. String I/O functions

Character I/O functions are functions that programme input/output of one character at a time. Since they deal with individual character values, these are the most basic I/O functions. To enter a character from the keyboard, use the following functions:

1. `getchar()`
2. `getch()`
3. `getche()`

Where as the output of a character as the monitor, the following functions can be used:

1. `putchar()`
2. `putch()`

Beside these, `getc()` and `putc()` can also be used for one same purpose.

Let us see the working of above mentioned console I/O functions with the help of programs.

`getchar()` and `putchar()`

`getchar()` function is used for reading a character from the keyboard. The syntax for the `getchar()` function is:

```
Character_variable = getchar(void);
```

Where `character_variable` is any valid C variable name. The word `void` indicates that no argument is needed for calling the function. Following statement reads a character and stores it in variable

`ch`, that is of type `char` obviously.

```
ch = getchar();
```

The `getchar()` waits for the character input until a character is typed at the keyboard. The typed character is echoed to the monitor and before assigning this value to the character variable appeared on the left side, it requires a carriage return (enter key) to be type by the user. `getchar()` function returns a value called EOF (End of File) if an error occurs.

Typically, the value of EOF is 1, though this may vary from compiler to compiler.

The `putchar()` is complementary function of `getchar()`. It is used to display a character on the monitor. The syntax for the `putchar()` function is:

```
putchar(character_variable);
```

Where `character_variable` refers to some previously declared character variable. The following statement displays a character value on the monitor whatever is stored inside `ch` at the current cursor position.

```
putchar(ch);
```

We can also use `putchar()` with character value directly, as shown below.

```
putchar('V');
```

This statement will display the character `V` as the monitor, whereas the statement

```
putchar('\n');
```

would cause the cursor on the screen to move to the beginning of the next line.

This function also returns EOF if there occurs an error.

The following program illustrates the working of these functions. This program will accept a character from the keyboard and will print it on the monitor screen



Example:

```
#include <stdio.h>

void main( )
{
    int c;
    printf("Enter a character");
    c = getchar();
    putchar(c);
}
```

6.8 gets() and puts() Functions

Character I/O functions have a drawback in that they can only handle one character at a time. Strings, on the other hand, are commonly used in real-world programmes. A string is nothing more than a set of characters. String I/O functions are functions that make it easier to move strings between a computer and regular I/O devices. Following function can be used for handling strings I/O:

1. gets()
2. puts()

gets() function is used to accept a string from the keyboard whereas puts() function is used to print a string on the monitor. Besides these I/O functions, C's standard library also provides several functions for various string handling operations. Let's first discuss gets() and puts() in this section.

The gets() function receives a sequence of characters i.e., a string entered at the keyboard and store them in a variable (essentially as Array of type char) mentioned with it.

gets() function

The gets() function allows the user to type a string of characters and then press the enter key. A character list is used to store all of the characters entered by the user. To render the array a string, the null character is inserted. The user will enter space-separated strings using the gets() method. It returns the string that the user typed in. The gets() function is problematic to use because it doesn't search for array bounds and keeps reading characters until the new line (enter) is encountered.

Declaration

```
char[ ] gets(char[]);
```



Example:

Program for gets ()

```
#include<stdio.h>

void main ()
{
    char s[25];
    printf("Enter the string? ");
    gets(s);
}
```

```
printf("You entered %s",s);
}
```

puts() function

The puts() function looks a lot like the printf() function. The puts() function is used to print the string that was previously read using the gets() or scanf() functions on the console. The integer value returned by the puts() function represents the number of characters printed on the console. Since it prints an extra newline character with the string, which transfers the cursor to a new line on the console, the integer value returned by puts() will always be the number of characters in the string plus 1.

Declaration

```
int puts(char[])
```



Example:

```
#include<stdio.h>
#include <string.h>
int main(){
char name[30];
printf("Enter your name: ");
gets(name); //reads string from user
printf("Your name is: ");
puts(name); //displays string
return 0;
}
```



Lab Exercise:

Program For gets() and puts()

```
#include <stdio.h>
void main( )
{
char name[10];
printf("Enter Name");
gets(name);
printf("Entered name is");
puts(name);
return 0;
}
```

Difference between Formatted and Unformatted Functions

- Formatted input and output functions contain format specifier in their syntax.
- Unformatted input and output functions do not contain format specifier in their syntax.
- printf() and scanf() are examples for formatted input and output functions.
- getchar(), gets(), puts(), putchar() etc. are examples of unformatted input output functions.

Function	Purpose
getchar()	Input exactly one character from keyboard
putchar()	Print exactly one character
gets()	Input collection of Characters
puts()	Print collection of Characters

**Lab Exercise:****Program**

```
#include<stdio.h>

void main()
{
    char str [11]; /* declar a character array str of size 11 */
    printf("\n enter a string (Maximum 10 characters):");
    gets(str); /* will read a string from the keyboard*/
    printf("\n the entered string \n:");
    puts(str); /* will print the under of str on the monitor and */
    print(" /* and advances the cursor to the .....*/ thank you");
}
```

Output:

Enter a string (maximum 10 characters): GoodDay

The entered string is: GoodDay

Thank you

Summary

- I/O operations deal with the transfer of data to peripheral devices such as monitor, key board, printer or secondary storage etc.
- A library is nothing more than one or more files that contain a group of predefined functions. In its most general form the word 'console' refers to the standard input and output devices.
- Unformatted console I/O functions doesn't allow input and output to be formatted as per the user requirements. getchar() function is used for reading a character from the keyboard.
- The putchar() is complementary function of getchar(). It is used to display a character on the monitor.
- The another possible use of getch() is to temporarily halt the execution of a program intentionally. gets() function is used to accept a string from the keyboard whereas puts() function is used to print a string on the monitor.

- Formatted I/O Functions allows input and output operations to be performed in a fixed format.
- The printf() is one of the most important and useful functions to display data on monitor.
- Except for double quotes, escape sequences and conversions specifiers, all characters within a pair of double quotes will be treated as literal text (string context) and will be displayed as it is on the monitor.
- Any character that is prefixed with a backslash is supposed to be treated as an escape sequence.
- Conversion specifiers always begin with the percent sign (%) and immediately followed by one or more conversion characters.

Keywords

printf()

scanf()

getchar()

putchar()

gets()

puts()

Header files: A text file that contains prototype of functions, definitions of constants etc. and which can be included in a C program file to access those functions and constants.

#include compiler directive: This compiler directive instructs the compiler to insert the contents of the specified file in place of this line.

Standard library: A group of in-built functions stored in a single file as a unit.

Self Assessment

1. Which format specifier is used for integer?
 - A. %c
 - B. %d
 - C. %f
 - D. %h
2. Which is the correct file pointer for standard input?
 - A. Stdout
 - B. Stdin
 - C. Stderr
 - D. Stdab
3. Which function is used for output on screen?
 - A. Main ()
 - B. Scanf ()
 - C. Printf ()
 - D. Get ()
4. What is the extension of a header file?
 - A. .c

- B. .b
 - C. .e
 - D. .h
5. `main()` is a
- A. Keyword
 - B. Function
 - C. Integer
 - D. Pointer
6. Which format specifier is used for float?
- A. % c
 - B. % d
 - C. % h
 - D. % f
7. Input is taken using function....
- A. `Printf()`
 - B. `Fprintf()`
 - C. `Scanf()`
 - D. `Main()`
8. `Putchat()` is used for..
- A. As input function for string
 - B. display exactly one character as output.
 - C. Display one string on screen
 - D. Input from user
9. `Getchar()` having similarities with..
- A. `Puts()`
 - B. `Gets()`
 - C. `Main()`
 - D. `Putchar()`

Answers for Self Assessment

- | | | | | |
|------|------|------|------|------|
| 1. B | 2. B | 3. C | 4. D | 5. B |
| 6. D | 7. C | 8. B | 9. B | |

Review Questions

1. Define `stdin`, `stdout`, and `stderr`.
2. Differentiate the followings:
(a) `printf()` and `puts()`

(b) `getche()` and `getch()`

(c) `scanf()` and `gets()`

3. How format string is associated with `printf()`? Discuss the various possible components of a format string in detail.

4. What happens if one uses variables in `scanf()` without using the address of operator (&)?

Discuss.

5. An amount of rupees, say R, is deposited in a bank for Y years, which pays simple interest at the rate of 'rt' annually. Write a C program that prints the amount after Y years

6. Write down two functions `xgets()` and `xputs()` which work similar to the standard library functions `gets()` and `puts()`.

7. What is the differences between `getchar()`, `fgetchar()`, `getch()` and `getche()`? With the help of suitable example.

8. Write down two functions `xgets()` and `xputs()` which work similar to the standard library functions `gets()` and `puts()`.

9. Write down a function `getint()`, which would receive a numeric string from the keyboard, convert it to an integer number and return the integer to the calling function. A sample usage of `getint()` is shown below:

```
main()
{
    int a ;
    a = getint() ;
    printf ( "you entered %d", a )
}
```

10. What is the differences between `getchar()`, `fgetchar()`, `getch()` and `getche()`?



Further Readings

Ashok N. Kamthane, "Programming with ANCI & Turbo C", Pearson Education, Year of Publication: 2008

B.W. Kernighan and D.M. Ritchie, "The Programming Language", Prentice Hall of India, New Delhi

Byron Gottfried, "Programming With C", Tata McGraw Hill Publishing Company Limited, New Delhi

R.G. Dromey, Englewood Cliffs, N.J., How to Solve it by Computer, Prentice-Hall International, 1982.

Yashvant Kanetkar, Let us C

Greg W Scragg, Genesco Suny, Problem Solving with Computers, Jones and Bartlett, 1997.